

An aerial photograph of a modern architectural complex in an urban setting. The complex features several multi-story white buildings with a grid-like window pattern. A prominent skybridge connects two of the buildings, spanning over a city street. The scene is captured during sunset, with a warm orange glow and a rainbow visible on the left side. In the background, a city skyline and a distant industrial smokestack are visible under a hazy sky.

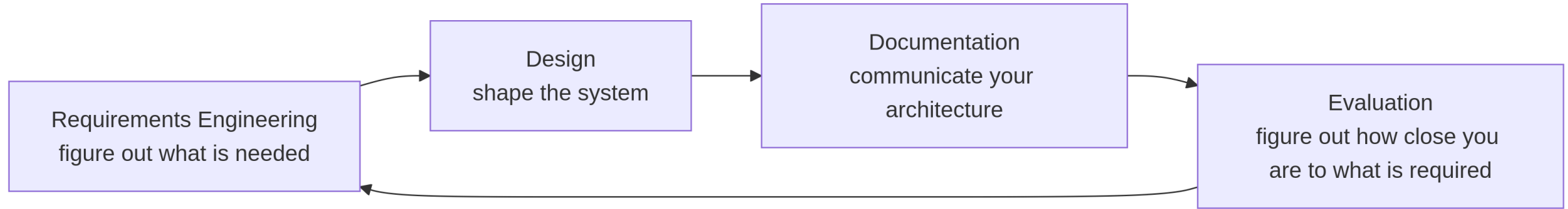
Methods of architecture

Table of contents

- Architecture life cycle
- Technical debts
- Solution strategy
- Risks

The architecture life cycle

A software architect is always working in one of the below phases. During their work on systems the steps illustrated by the picture below are traversed.



Note: The length of an iteration depends on the system, project and processes. It can be months, weeks, days or hours.

The Architecture Life Cycle (Phases)

The software architecture lifecycle consists of four key phases. Whenever an architect is doing something the task can be assigned to one of these phases:

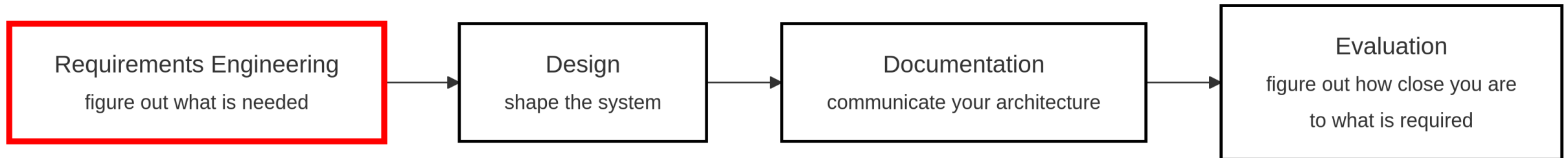
- **Requirements Engineering**
figure out what is needed
- **Design**
shape the system
- **Documentation**
communicate your architecture
- **Evaluation**
figure out how close you are to what is required

Phase 1: Requirements Engineering

Every project starts by understanding what the customer wants.

Key Insight: It's rarely possible to extract a complete set of requirements initially, but you must establish:

- Basic understanding of the problem field (domain)
- Stakeholder's wishes, expectations and motivation
- Goals of the project
- Clear understanding of quality requirements



Phase 2: Design

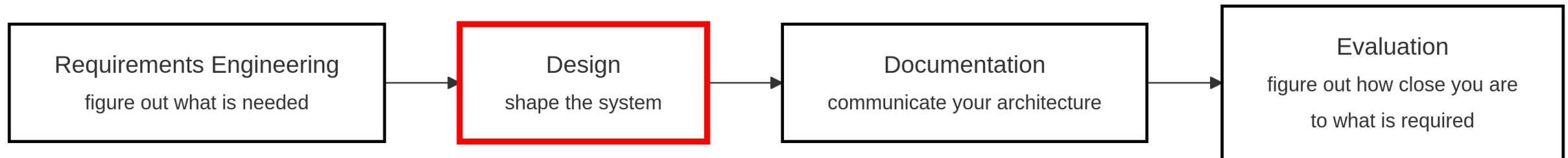
In this phase you are shaping (or at least parts) the system.

Design Phase Activities:

- Define business context and main system components
- Establish interactions between components
- Set architectural patterns and styles

Note: The degree of details depends on Organizational culture, Project environment and Development team skills.

Note: In agile projects big upfront designs (**BUFD**) are typically avoided.



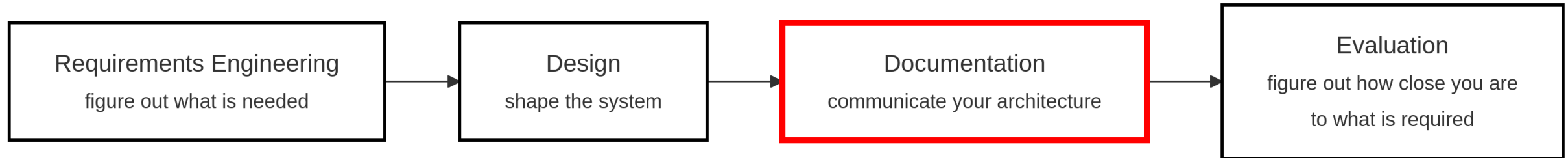
Phase 3: Documentation (Communication)

After shaping the architecture in the design phase, documentation is created and necessary communication takes place.

Documentation Purposes:

- Guide developers during implementation
- Provide abstract and particular documentation for stakeholders

Key Benefit: Enables communication and shared understanding across the team.



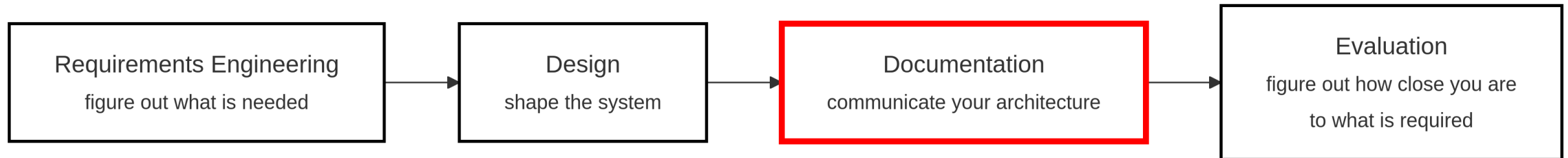
Phase 4: Evaluation

Finally you can assess how well the system satisfies customer needs.

Evaluation Focus:

- Measure alignment with requirements
- Identify gaps or issues
- Validate architectural decisions
- Get feedback from operations from production

Note: Various evaluation methods exist but are beyond the scope of this lectures.



Life Cycle Considerations

Key Principles:

1. Every **work product** and **activity** belongs to one of these phases
2. Phases have **information dependencies** - you need predecessor phase information
3. Steps are **not always strictly sequential** and often executed multiple times
4. Different system parts may follow **separate life cycles**

Agile Context: Multiple iterations are common before system parts are completed.

Practical Application

Categorizing by Phase:

- Organizes architecture work into a structured overview
- Clarifies phase-specific activities and deliverables
- Supports process definition and improvement

Common Practice: Specify a concrete architecture process tailored to project needs and make this part of your solution strategy.

Summary: Architecture Life Cycle

Four Interconnected Phases:

1. **Requirements Engineering** - Understand needs
2. **Design** - Shape the system structure
3. **Documentation** - Communicate the architecture
4. **Evaluation** - Validate against requirements

Remember: This is a conceptual framework that guides but doesn't dictate implementation. Adapt it to your project's specific context and methodology.

Solution Strategy, what is part of it

During a project the architect has to manifest a particular solution strategy.
This involves:

- technology stack (programming languages)
- tactics for achieving quality goals (reliability through unit testing)
- organizations decisions like teams, process (SCRUM), stakeholder management
- basic decisions about architecture styles
- decisions about design/architecture patterns
- ...

Solution Strategy, documentation

A structured documentation of the solution strategy could be done as a table:

Quality goal	Scenario	Solution approach	Link to Details
Maintainability	<i>Integrate a new payment provider to the system within one team sprint.</i>	<i>Create an exchangeable payment provider adapter that handles all dependencies to accounting and head of payment.</i>	<i>Link to some UML diagram of the adapter.</i>
<Q-goal 2>	<Text>	<Text>	<Link>

Solution Strategy, justification

Decisions made in the solution strategy have to be justified.

The “why” is often more important than the “what” or “how”.

HINT: Existing examples are good arguments for convincing non technical people. For instance, if you want to convince c-level for buying Kubernetes you can create a list of other companies who are already migrated successfully. This is also one reason why networking between software architects is important.

Technical Debt

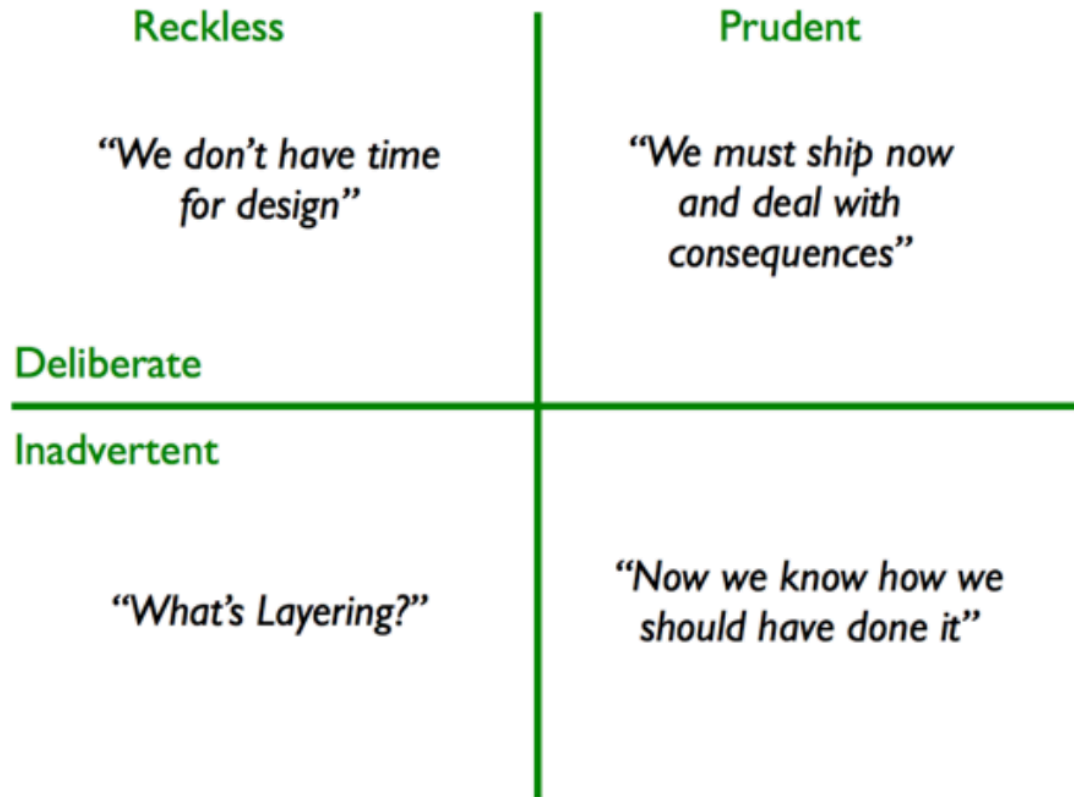
Technical debt is a metaphor for the implied cost of additional rework caused by choosing an easy or limited solution now instead of using a better approach that would be more effort or time consuming.

Key Characteristics:

- Accumulates when quick fixes are prioritized over proper solutions
- Can be deliberate (strategic) or accidental (unintentional)
- Often impacts maintainability, security, scalability, and team velocity so that it has to be reworked properly

HINT: Technical debt isn't inherently bad—it's often a necessary trade-off for faster delivery. The problem arises when it becomes unmanaged and compounds over time, making future changes exponentially more difficult.

Technical dept quadrant, the four categories



1. Reckless & Deliberate: "We don't have time for design"
2. Prudent & Deliberate: "We must ship now and deal with consequences"
3. Reckless & Inadvertent: "What's layering?" (due to ignorance)
4. Prudent & Inadvertent: "Now we know how we should have done it"

Technical dept, handling

There are effective strategies in place to get an handle on technical depts.

1. **Acknowledge & Track:** Make debt visible in your backlog
2. **Prioritize Ruthlessly:** Not all debt needs immediate rework
3. **Allocate Time:** Dedicate 10-20% of each sprint to debt reduction (**maintenance share**)
4. **Prevent Accumulation:** Do code reviews and set architectural principles
5. **Measure & Monitor:** Track debt-related metrics (code complexity, bug counts, etc.)

Technical Debt, the backlog:

- Create a dedicated section in your product backlog
- Explain the costs a technical debt is causing
- Review and prioritize the backlog regularly during architecture refinement sessions
- Remove items that are residing for longer time with low priority in the backlog

HINT: Treat technical debt like financial planning—sometimes taking on debt is strategic, but you need a repayment plan. Regular "debt reviews" with your team ensure it doesn't spiral out of control.

Risks

Similar to a project manager an architect should always have potential risks in mind. The question "What could cause my system/project to fail ?" should be answered upfront.

HINT: Create the top ten list of risks that could fail your project.

Note: Technical debts are very often introducing risks to a project. It is

Note: Potential risks need to be communicated to stakeholders on a regular basis.